10. Процедурно програмиране – основни информационни и алгоритмични структури (C++).

Основните принципи на структурното програмиране са принципа за **модулност** и принципа за **абстракция на данните**.Съгласно принципа за модулност, програмата се разделя наподходящи взаимосвързани части, всяка от които се реализирачрез определени средства. Целта е промените в представянето наданните да не променят голям брой от модулите на програмата. Съгласно принципа за абстракция на данните, методите за използване на данните се отделят от методите за тяхното конкретно представяне. Програмите се конструират така, че да работят с абстрактни данни – данни с неуточнено представяне. След това представянето се конкретизира с помощта на множество функции, наречени конструктори, мутатори и функции за достъп (селектори), които реализират абстрактните данни по конкретен начин. Така при решаването на даден проблем се оформят следните нива на абстракция:
1. Приложения в проблемната област.
2. Модули, реализиращи основните операции над данните.
3. Примитивни операции – конструктори, мутатори, селектори.
4. Избор на представянето на данните. Реализацията на подхода трябва да е такава, че всяко ниво използва единствено средствата на непосредствено следващото го ниво. По този начин, промените, които възникват на едно ниво ще се отразят само на предходното ниво. При изпълнение на компютърна програма се извършват определени действия над данните, дефинирани в програмата. Тези данни се съхраняват в **информационните структури**, допустими в съответния език за програмиране. Най-общо типовете информационни структури могат да бъдат разделени на два вида: вградени и абстрактни. Вградените типове са предварително дефинирани и се поддържат от самия език, а абстрактните типове се дефинират от програмиста. Друга класификация на типовете е следната: скаларни и съставни типове.

1.) Скаларните типове представят данни, които се състоят само от една компонента. При съставните типове данни, данните представляват редица от компоненти. Скаларните типове, поддържани в C++ са следните: булев, цял, реален, символен, изброен, указател, псевдоним. Съставните типове, поддържани в C++ са следните: масив, вектор, запис.

Логически тип

Нарича се още булев тип. Типът е стандартен, вграден в реализацията на C++. За означаването му се използва запазената дума bool (съкращение от boolean).

Множество от стойности

Състои се от два елемента – стойностите true (истина) и false (лъжа). Тези стойности се наричат още булеви константи.

<булева_константа> ::= true | false.

Променлива величина, множеството от допустимите стойности, на която съвпада с множеството от стойности на типа булев, се нарича булева или логическа променлива или променлива от тип булев. Дефинира се по обичайния начин.

Примери:
bool b1, b2;
bool b3 = false;

Дефиницията свързва булевите променливи с множеството от стойности на типа булев или с конкретна стойност от това множество като отделя по 1 байт оперативна памет за всяка от тях. Стойността на тази памет е неопределена или е булевата константа, свързана с дефинираната прменлива, в случай, че тя е инициализирана.

Оператори и вградени функции

Логически оператори

Оператор за логическо умножение (конюнкция)
Той е двуаргументен (бинарен) оператор. Означава се с and или && (за Visual C++)
Операторът се поставя между двата си аргумента. Такива оператори се наричат инфиксни.

Оператор за логическо събиране (дизюнкция)
Той също е бинарен, инфиксен оператор. Означава се с or или || (за Visual C++ 6.0)
Оператор за логическо отрицание

Той е едноаргументен (унарен) оператор. Означава се с not или ! (за Visual C++ 6.0)
Поставя се пред единствения си аргумент. Такива оператори се наричат префиксни.
Смисълът на операторите and, or и not е разширен чрез разширяване смисъла на булевите константи. Прието е, че true е всяка стойност, различна от 0 и че false е стойността 0.

Оператори за сравнение

Над булевите данни могат да се извършват следните инфиксни оператори за сравнение:
==, !=, >, >=, <, <=
Сравняват се кодовете.
Примери:
false < true e true
false > false e false
true >= false e true

Въвеждане

Не е възможно въвеждане на стойност на булева променлива чрез оператора >>, т.е. операторът
cin >> b1;
където b1 е булевата променлива, дефинирана по-горе, е недопустим.

Извеждане

Осъществява се чрез оператора
cout << <булева_константа>;
или по-общо
cout << <булев_израз>;
където синтактичната категория <булев_израз> е определена по-долу.
Извежда се кодът на булевата константа или кодът на булевата константа, която е стойност на <булев_израз>.

Булеви изрази

Булевите изрази са правила за получаване на булева стойност. Дефинират се рекурсивно по следния начин:

Булевите константи са булеви изрази.
Булевите променливи са булеви изрази.
Прилагането на булевите оператори not (!), and (&&), or (||) над булеви изрази е булев израз.
Прилагането на операторите за сравнение ==, !=, >, >=, <, <= към булеви изрази е булев израз.

Числени типове

Целочислени типове

Ще разгледаме целочисления тип int.

Типът е стандартен, вграден в реализацията на езика. За означаването му се използва запазената дума int (съкращение от integer).

Множество от стойности

Множеството от стойности на типа int зависи от хардуера и реализацията и не се дефинира от ANSI (American National Standarts Institute). Състои се от целите числа от някакъв интервал. За реализацията Visual C++ 6.0, това е интервалът [-231, 231-1].
Целите числа се записват като в математиката, т.е.
<цяло_число> ::= [+|-]<цяло_без_знак>
<цяло_без_знак> ::= <цифра>|<цифра><цяло_без_знак>
<цифра> ::= 0|1| … |9.
Обикновено знакът + се пропуска.
Допълнение: Целите числа могат да са в десетична, осмична и шестнадесетична позиционна система. Осмичните числа започват с 0 (нула), а шестнадесетичните - с 0x (нула, x).

Елементите от множеството от стойности на типа int се наричат константи от тип int. Променлива величина, множеството от допустимите стойности, на която съвпада с множеството от стойности на типа int, се нарича цяла променлива или променлива от тип int. Дефинира се по обичайния начин. Дефиницията свързва променливата с множеството от стойности на типа int или с конкретна стойност от това множество като отделя по 4 байта (1 дума) оперативна памет за всяка от тях. Ако променливата не е била инициализирана, стойността на свързната с нея памет е неопределена, а в противен случай е указаната при инициализацията стойност.

Аритметични оператори

Унарни оператори

Записват се пред или след единствения си аргумент.

+, - са префиксни оператори. Потвърждават или променят знака на аргумента си.

Следните означения съдържат унарния оператор + или -:

-i +j -j +i -567

Бинарни оператори

Имат два аргумента. Следните аритметични оператори са инфиксни:

Оператор	Операция
+	събиране
-	изваждане
*	умножение
/	целочислено деление
%	остатък от целочислено деление

Логически оператори

Логическите оператори, реализиращи конюнкция, дизюнкция и отрицание, могат да се прилагат над целочислени константи. Дефинират се по същия начин, както при булевите константи, но целите числа, които са различни от 0 се интерпретират true, а 0 – като false.
Примери:
123 and 0 e false
0 or 15 e true
not 67 e false

Оператори за сравнение

Над цели константи могат да се прилагат следните инфиксни оператори за сравнение:

Оператор	Операция
==	сравнение за равно
!=	сравнение за различно
>	сравнение за по-голямо
>=	сравнение за по-голямо или равно
<	сравнение за по-малко
<=	сравнение за по-малко или равно

Наредбата на целите числа е като в математиката.

Вградени функции

В езика C++ има набор от вградени функции. Обръщението към такива функции има следния синтаксис:

<име_на_функция>(<израз>, <израз>, …, <израз>)

и връща стойност от типа на функцията.

Тук ще разгледаме само едноаргументната целочислена функция abs.

abs(x) – намира |x|, където x е цял израз

Примери:

abs(-1587) = 1587 abs(0) = 0 abs(23) = 23

Въвеждане

Реализира се по стандартния и разгледан вече начин.
Пример: Ако
int i, j;
операторът
cin >> i >> j;
въвежда стойности на целите променливи i и j.

Извеждане

Реализира се чрез оператора
cout << <цяла_константа>;

Други целочислени типове
Други цели типове се получават от int като се използват модификаторите short, long, signed и unsigned. Тези модификатори доопределят някои аспекти на типа int.
За реализацията Visual C++ 6.0 са в сила:

Тип	Диапазон	Необходима памет
short int	-32768 до 32767	2 байта
unsigned shortint	0 до 65535	2 байта
long int	-2147483648 до 2147483647	4 байта
unsigned long int	0 до 4294967295	4 байта
unsigned int	0 до 4294967295	4 байта

Запазената дума int при тези типове се подразбира и може да бъде пропусната. Типовете short int (или само short) и long int (или само long) са съкратен запис на signed short int и signed long int.

Реални типове

Ще разгледаме реалния тип double.
Типът е стандартен, вграден във всички реализации на езика.

Множество от стойности

Множеството от стойности на типа double се състои от реалните числа от -1.74*10308 до 1.7*10308 . Записват се във два формата – като числа с фиксирана и като числа с плаваща запетая (експоненциален формат).

<реално_число> ::= <цяло_число>.<цяло_число_без_знак>|
<цяло_число>E<порядък>|

<цяло_число>.<цяло_число_без_знак>E<порядък>|
<порядък> ::= <цяло_число>
При експоненциалния формат може да се използва и малката латинска буква e.
Примери: Следните числа
123.3454 -10343.034 123E13 -1.23e-4
са коректно записани реални числа. Смисълът на експоненциалния формат е реално число, получено след умножаване на числото пред E (e) с 10 на степен числото след E (e).
Примери: 12.5E4 е реалното число 125000.0, а –1234.025e-3 е реалното число –1.234025.
Дефинира се по обичайния начин. Заделят се по 8 байта оперативна памет за всяка променлива от тип double
Аритметични оператори

Унарни оператори

+, - Префиксни са. Потвърждават или променят знака на аргумента си.

Бинарни оператори

Имат два аргумента. Следните аритметичните оператори са инфиксни:

Оператор	Операция
+	събиране
-	изваждане
*	умножение
/	деление (поне единият аргумент е реален)

Логически оператори

Логическите оператори, реализиращи операциите конюнкция, дизюнкция и отрицание, могат да се прилагат над реални константи. Дефинират се по същия начин, както при булевите константи, като реалните числа, които са различни от 0.0 се интерпретират като true, а 0.0 – като false.

Оператори за сравнение

Над реални данни могат да се прилагат стандартните инфиксни оператори за сравнение (==, !=, >, >=, <, <=).

Допълнение: Сравнението за равно на две реални числа x и y се реализира обикновено чрез релацията: $|x - y| < \varepsilon$, където $\varepsilon = 10\text{-}14$ за тип double. По-добър начин е да се използва релацията:

Вградени функции

$$\frac{|x - y|}{\max\{|x|, |y|\}} \leq \varepsilon$$

При цял или реален аргумент, следните функции връщат реален резултат от тип double:

Функция	Намира
sin(x)	Синус, sin x, x е в радиани
Cos(x)	косинус, cos x, x е в радиани
tan(x)	тангенс, tg x, x е в радиани
asin(x)	аркуссинус, arcsin $x \in [-\pi/2, \pi/2]$, $x \in [-1, 1]$
acos(x)	аркускосинус, arccos $x \in [0, \pi]$, $x \in [-1, 1]$
atan(x)	аркустангенс, arctg $x \in (-\pi/2, \pi/2)$
exp(x)	експонента, ex
log(x)	натурален логаритъм, ln x, x > 0
log10(x)	десетичен логаритъм, lg x, x > 0
sinh(x)	хиперболичен синус, sh x
cosh(x)	хиперболичен косинус, ch x
tanh(x)	хиперболичен тангенс, th x
ceil(x)	най-малкото цяло ≥ x, преобразувано в тип double

Floor(x)	най-голямото цяло ≤ x, преобразувано в тип double
fabs(x)	абсолютна стойност на x, |x|
sqrt(x)	Корен квадратен от x, x ≥ 0
pow(x, n)	степенуване, xn (x и n са реални от тип double).

Въвеждане и Извеждане се Реализира по стандартния начин операторите cin и cout.
Допълнение: за допълнително форматиране на изхода е възможно да се използват манипулаторите: setw (Задава широчината на следващото изходно поле.) и setprecision () задава броя на цифрите, използвани при извеждане на следващите реални числа, а в съчетание с setiosflags(ios::fixed), задава броя на цифрите след десетичната точка на извежданите реални числа.
int x = 21.5632
Операторът

```
cout << setiosflags(ios::fixed)
 << setprecision(3)
 << setw(10) << x << "\n";
```

извежда
****21.563
Други реални типове

В езика C++ има и друг реален тип, наречен float. Различава се от типа double по множеството от стойностите си и заеманата памет.
Множеството от стойности на типа float се състои от реалните числа от диапазона от – 3.4*1038 до 3.4*1038. За записване на константите от този диапазон са необходими 4 байта ОП.

Структура от данни масив

Под структура от данни се разбира организирана информация, която може да бъде описана, създадена и обработена с помощта на програма.

За да се определи една структура от данни е необходимо да се направи:
- логическо описание на структурата, което я описва на базата на декомпозицията ѝ на по-прости структури, а също на декомпозиция на операциите над структурата на по-прости операции.
- физическо представяне на структурата, което дава методи за представяне на структурата в паметта на компютъра.

Елементите на структури, които се състоят от една компонента, се наричат прости, или скаларни. Структури от данни, компонентите на които са редици от елементи, се наричат съставни.
Структури от данни, за които операциите включване и изключване на елемент не са допустими, се наричат статични, в противен случай - динамични.
Логическо описание

Масивът е крайна редица от фиксиран брой елементи от един и същ тип. Към всеки елемент от редицата е възможен пряк достъп, който се осъществява чрез индекс. Операциите включване и изключване на елемент в/от масива са недопустими, т.е. масивът е статична структура от данни.

Физическо представяне

Елементите на масива се записват последователно в паметта на компютъра, като за всеки елемент на редицата се отделя определено количество памет.

В езика C++ структурата масив се реализира чрез типа масив.
Тип масив

В C++ структурата от данни масив е реализирана малко ограничено. Разглежда се като крайна редица от елементи от един и същ тип с пряк достъп до всеки елемент, осъществяващ се чрез индекс с цели стойности, започващи от 0 и нарастващи с 1 до указана горна граница. Дефинира се от програмиста.

Дефиниране на масив

Типът масив се определя чрез задаване на типа и броя на елементите на редицата, определяща масив. Нека T е име или дефиниция на произволен тип, различен от псевдоним, void и функционален. За типа T и константния израз от интегрален или изброен тип с положителна стойност size, T[size] е тип масив от size елемента от тип T. Елементите се индексират от 0 до size–1. T се нарича базов тип за типа масив, а size – горна граница.

Примери:
int[5] дефинира масив от 5 елемента от тип int, индексирани от 0 до 4;
double[10] дефинира масив от 10 елемента от тип double, индексирани от 0 до 9;
bool[4] дефинира масив от 4 елемента от тип bool, индексирани от 0 до 3.
Множество от стойности

Множеството от стойности на типа T[size] се състои от всички редици от по size елемента, които са произволни константи от тип T. Достъпът до елементите на редиците е пряк и се осъществява с помощта на индекс, като достъпът до първия елемент се осъществява с индекс със стойност 0, до последния – с индекс със стойност size-1, а до всеки от останалите елементи – с индекс със стойност с 1 по-голяма от тази на индекса на предишния елемент. Елементите от множеството от стойности на даден тип масив са константите на този тип масив.
Примери:
1. Следните редици {1,2,3,4,5}, {-3, 0, 1, 2, 0}, {12, -14, 8, 23, 1000} са константи от тип int[5].

2. Редиците {1.5, -2.3, 3.4, 4.9, 5.0, -11.6, -123.56, 13.7, -32.12, 0.98}, {-13, 0.5, 11.9, 21.98, 0.03, 1e2, -134.9, 0.09, 12.3, 15.6} са константи от тип double[10].
Дефиниция на масив

<дефиниция_на_променлива_от_тип_масив> ::=
T <променлива>[size] [= {<редица_от_константни_изрази>}]опц
{,<променлива>[size] [= {<редица_от_константни_изрази>}]опц }опц;

където

- T е име или дефиниция на произволен тип, различен от псевдоним, void, функционален;
- <променлива> е идентификатор;
- size е константен израз от интегрален или изброен тип с положителна стойност;
- <редица_от_константни_изрази> се дефинира по следния начин:

<редица_от_константни_изрази> ::= <константен_израз>|
<константен_израз>, <редица_от_константни_изрази>

като константните изрази са от тип T или от тип, съвместим с него.
Примери:

```
int a[5];
double c[10];
bool b[3];
enum {FALSE, TRUE} x[20];
double p[4] = {1.25, 2.5, 9.25, 4.12};
```

Дефиницията
T <променлива>[size] = {<редица_от_константни_изрази>}
се нарича дефиниция на масив с инициализация, а Фрагмента
{<редица_от_константни_изрази>} - инициализация. При нея е възможно size да се пропусне. Тогава за стойност на size се подразбира броят на константните изрази, изброени в инициализацията. Ако size е указано и изброените константни изрази в инициализацията са по-малко от size, останалите се приемат за 0.

Примери:

Дефиницията
int q[5] = {1, 2, 3};
е еквивалентна на
int q[] = {1, 2, 3, 0, 0};
Дефиницията
double r[] = {0, 1, 2, 3};
е еквивалентна на
double r[4] = {0, 1, 2, 3};

Дефиницията с инициализация е един начин за свързване на променлива от тип масив с конкретна константа от множеството от стойности на този тип масив. Друг начин предоставят т.нар. индексирани променливи. С всяка променлива от тип масив е свързан набор от индексирани променливи. Фиг. 6.2 илюстрира техния синтаксис.

Синтаксис на индексирани променлии
<индексирана_променлива> ::=
<променлива_от_тип_масив>[<индекс>]
където
<индекс> е израз от интегрален или изброен тип.
Всяка индексирана променлива е от базовия тип.

Синтаксис на индексираните променливи

Примери:
1. С променливата a, дефинирана по-горе, са свързани индексираните променливи a[0], a[1], a[2], a[3] и a[4], които са от тип int.
2. С променливата b са свързани индексираните променливи b[0], b[1],…, b[9], които са от тип double.
3. С променливата x са свързани индексираните променливи x[0], x[1],…, x[19], които са от тип enum {FALSE, TRUE}.

Дефиницията на променлива от тип масив не само свързва променливата с множеството от стойности на указания тип, но и отделя определено количество памет (обикновено 4B), в която записва адреса в паметта на първата индексирана променлива на масива. Останалите индексирани променливи се разполагат последователно след

първата. За всяка индексирана променлива се отделя по толкова памет, колкото базовият тип изисква.
Операции и вградени функции

Не са възможни операции над масиви като цяло, но всички операции и вградени функции, които базовият тип допуска, са възможни за индексираните променливи, свързани с масива.

Пример: Нека

int a[5], b[5];

Недопустими са:

cin >> a >> b;

a = b;

а също a == b или a != b.

Операторът

cout << a;

е допустим и извежда адреса на a[0].
Някои приложения на структурата от данни масив са: Търсене на елемент в редица, Сортиране на редица, Сливане на редици (за повече инфо за писмения изпит - вж. учебника)
Друго приложение на типът масив е за представяне на символни низове в C++.
Разглежданите досега масиви се наричат едномерни. Те реализират крайни редици от елементи от скаларен тип. Възможно е обаче типът на елементите да е масив. В този случай се говори за многомерни масиви.

Многомерни масиви

Масив, базовият тип на който е едномерен масив, се нарича двумерен. Масив, базовият тип на който е двумерен масив, се нарича тримерен и т.н. На практика се използват масиви с размерност най-много 3.

Дефиниране на многомерни масиви

Нека T е име или дефиниция на произволен тип, различен от псевдоним, void и функционален, size1, size2, …, sizen (n>1 е дадено цяло число) са константни изрази от интегрален или изброен тип с положителни стойности. T[size1][size2] … [sizen] е тип n-мерен масив от тип T. T се нарича базов тип за типа масив.

Примери:

int [5][3] дефинира двумерен масив от тип int;

double [4][5][3] дефинира тримерен масив от тип double;

Множество от стойности

Множеството от стойности на типа T[size1][size2] … [sizen] се състои от всички редици от по size1 елемента, които са произволни константи от тип T[size2] … [sizen].
Достъпът до елементите на редиците е пряк и се осъществява с помощта на индекс, като достъпът до първия елемент се осъществява с индекс със стойност 0, до последния – с индекс със стойност size1-1, а до всеки от останалите елементи – с индекс със стойност с 1 по-голяма от тази на индекса на предишния елемент. Елементите от множеството от стойности на даден тип многомерен масив са константите на този тип масив.
Примери:

Множеството от стойности на типа int[5][3] се състои от всички редици от по 5 елемента, които са едномерни масиви от тип int[3]. Достъпът до елементите на редиците се осъществява с индекс със стойности 0, 1, 2, 3 и 4.

Синтаксис на многомерен масив

Примери:

```
int x[10][20];
double y[20][10][5];
int z[3][2] = {{1, 3},
{5, 7},
{2, 9}};
int t[2][3][2] = {{{1, 3}, {5, 7}, {6, 9}},
        {{7, 8}, {1, 8}, {-1, -4}}};
```

Дефиницията на променлива от тип многомерен масив не само свързва променливата с множеството от стойности на указания тип, но и отделя определено количество памет за разполагане на индексираните променливи, свързани с нея. Индексираните променливи се разполагат последователно по по-бързото нарастване на по-далечните си индекси. За всяка индексирана променлива се отделя толкова памет, колкото базовият тип изисква.

Забележка: Двумерните масиви разполагат в ОП индексираните си променливи по по-бързото нарастване на втория индекс. Това физическо представяне се нарича представяне по редове. Тези масиви могат да бъдат използвани за реализация и работа с матрици и др. правоъгълни таблици.

Важно допълнение: При работа с масиви трябва да се има предвид, че повечето реализации не проверяват дали стойностите на индексите са в рамките на границите, зададени при техните дефиниции. Тази особеност крие опасност от допускане на трудноотриваеми грешки.

Тип указател

Променлива, това е място за съхранение на данни, което може да съдържа различни стойности. Идентифицира се с дадено от потребителя име (идентификатор). Има си и тип. Дефинира се като се указват задължително типът и името й. Типът определя броя на байтовете, в които ще се съхранява променливата, а също и множеството от операциите, които могат да се изпълняват над нея. Освен това, с променливата е свързана и стойност – неопределена или константа от типа, от който е тя. Нарича се още rvalue. Мястото в паметта, в което е записана rvalue, се идентифицира с адрес, който се нарича адрес на променливата или lvalue. По-точно адресът е адреса на първия байт от множеството байтове, отделени за променливата.

Пример: Фрагментът

```
int i = 1024;
```

дефинира променлива с име i и тип int. Стойността й (rvalue) е 1024. i именува място от паметта (lvalue) с размери 4 байта, като lvalue е адреса на първия байт на това място.

Намирането на адреса на дефинирана променлива става чрез унарния префиксен дясно-асоциативен оператор & (амперсанд). Приоритетът му е същия като на унарните оператори +, -, !, ++, -- и др.

Оператор &

Синтаксис

&<променлива>

където <променлива> е вече дефинирана променлива.

Семантика

Намира адреса на <променлива>.

Пример: &i е адреса на променливата i и може да се изведе чрез оператора cout << &i;

Операторът & не може да се прилага върху константи и изрази, т.е. &100 и &(i+5) са недопустими обръщения. Не е възможно също прилагането му и върху променливи от тип масив, тъй като те имат и смисъла на константни указатели.

Адресите могат да се присвояват на специален тип променливи, наречени променливи от тип указател или само указатели.

Дефиниране

Нека T е име или дефиниция на тип. За типа T, T* е тип, наречен указател към T. T се нарича указван тип или тип на указателя.

Примери:

int* е тип указател към тип int;

enum {a, b, c}* е тип указател към тип enum {a, b, c}.

Множество от стойности

Състои се от адресите на данните от тип T, дефинирани в програмата, преди използването на T*. Те са константите на типа T*. Освен тях съществува специална константа с име NULL, наречена нулев указател. Тя може да бъде свързвана с всеки указател независимо от неговия тип. Тази константа се интерпретира като "сочи към никъде", а в позиция на предикат е false.

Променлива величина, множеството от допустимите стойности, на която съвпада с множеството от стойности на типа T*, се нарича променлива от тип T*, променлива от тип указател към тип T или само указател към тип T. Дефинира се по общоприетия начин.

Дефиниция на променлива от тип указател

T* <променлива> [= <стойност>]опц

{,<променлива> [= <стойност>]опц}опц|

T *<променлива> [= <стойност>]опц

{*<променлива> [= <стойност>]опц}опц;

където

- T е име или дефиниция на тип;
- <променлива> е идентификатор;
- <стойност> е шестнадесетично число, представляващо адрес на данна от тип T или NULL.

В първия случай на дефиницията обаче има особеност – операторът * се свързва с T само за първата променлива. Дефиницията

T* a, b;

е еквивалентна на

T* a;

T b;

т.е. само променливата a е от тип указател към тип T.

Примери: Дефиницията

int *pint1, *pint2;

задава два указателя към тип int, а

```
int *pint1, pint2;
```

- указател pint1 към int и променлива pint2 от тип int.

Дефиницията на променлива от тип указател предизвиква в ОП да се отделят 4B, в които се записва някакъв адрес от множеството от стойности на съответния тип, ако дефиницията е с инициализация и неопределено или NULL, ако дефиницията не е с инициализация. Този адрес е стойността на променливата от тип указател, а записаното на този адрес е съдържанието й.

Пример: В резултат от изпълнението на дефинициите

```
int i = 12;
int* p = &i;              // p е инициализирано с адреса на i
double *q = NULL; // q е инициализирано с нулевия указтел
double x = 1.56;
double *r = &x;           // r е инициализирано с адреса на x
```

Операции и вградени функции

Извличане на съдържанието на указател

Осъществява се чрез префиксния, дясноасоциативен унарен оператор *. * има приоритет на унарен оператор.

Оператор *

Синтаксис

*<променлива_от_тип_указател>

Семантика

Извлича стойността на адреса, записан в <променлива_от_тип_ указател>, т.е. съдържанието на <променлива_от_тип_ указател>.

Като използваме дефинициите от примера по-горе, имаме:

*p е 12// 12 е съдържанието на p

```
*r е 1.56        // 1.56 е съдържанието на r
```

Освен, че намира съдържанието на променлива от тип указател, обръщението

*<променлива_от_тип_указател>

е променлива от тип T. (Всички операции, допустими за типа T, са допустими и за нея.

Като използваме дефинициите от примера по-горе, *p и *r са цяла и реална променливи, съответно. След изпълнение на операторите за присвояване

```
*p = 20;
*r = 2.18;
```

стойността на i се променя на 20, а тази на r – на 2.18.

Аритметични и логически операции

Променливите от тип указател могат да участват като операнди в следните аритметични и логически операции +, -, ++, --, ==, !=, >, >=, < и <=. Изпълнението на аритметични операции върху указатели е свързано с някои особености, заради което аритметиката с указатели се нарича още адресна аритметика. Особеността се изразява в т. нар. мащабиране. Ще го изясним чрез пример.

Да разгледаме фрагмента

```
int *p;
double *q;
...
p = p + 1;
q = q + 1;
```

Операторът p = p + 1; свързва p не със стойността на p, увеличена с 1, а с p + 1*4, където 4 е броя на байтовете, необходими за записване на данна от тип int (p е указател към int). Аналогично, q = q + 1; увеличава стойността на q не с 1, а с 8, тъй като q е указател към double (8 байта са необходими за записване на данна от този тип).
Общото правило е следното: Ако p е указател от тип T*, p+i е съкратен запис на p + i*sizeof(T), където sizeof(T) е функция, която намира броя на байтовете, необходими за записване на данна от тип T.

Въвеждане

Не е възможно въвеждане на данни от тип указател чрез оператора cin. Свързването на указател със стойност става чрез инициализация или оператора за присвояване.

Извеждане

Осъществява се по стандартния начин - чрез оператора cout.

Допълнение

Типът, който се задава в дефиницията на променлива от тип указател, е информация за компилатора относно начина, по който да се интерпретира съдържанието на указателя. В контекста на последния пример *p са четири байта, които ще се интерпретират като цяло число от тип int. Аналогично, *q са осем байта, които ще се интерпретират като реално число от тип double.
В някои случаи е важна стойността на променливата от тип указател (адресът), а не нейното съдържание. Тогава тя се дефинира като указател към тип void. Този тип указатели са предвидени с цел една и съща променлива - указател да може в различни моменти да сочи към данни от различен тип. В този случай, при опит да се използва съдържанието на променливата от тип указател, ще се предизвика грешка.
Съдържанието на променлива - указател към тип void може да се извлече само след привеждане на типа на указателя (void*) до типа на съдържанието. Това може да се осъществи чрез операторите за преобразуване на типове.
Пример:

```
int a = 100;
void* p;                                    // дефинира указател към void
p = &a;                                     // инициализира p
cout << *p;                                 // грешка
cout << *((int*) p);            // преобразува p в указател към int
// и тогава извлича съдържанието му.
```

В C++ е възможно да се дефинират указатели, които са константи (T* const <идентификатор>), а също и указатели, които сочат към константи (const T* <идентификатор>). И в двата случая се използва запазената дума const, която се поставя пред съответните елементи от дефинициите на указателите. Стойността на елемента, дефиниран като const (указателя или обекта, към който сочи) не може да бъде променяна. В дефиницията:

T* const <идентификатор>;

<идентификатор> е константен указател към тип T и не може да бъде променяна стойността му. В дефиницията:

const T* <идентификатор>;

<идентификатор> е указател към константа от тип T и не може да бъде променяно съдържанието му.
Пример:

```
int i, j = 5;
int *pi;                                    // pi е указател към int
```

```
int * const b = &i;      // b е константен указател към int
const int *c = &j;       // c е указател към цяла константа.
b = &j;                                    // грешка, b е константен указател
*c = 15;                                   // грешка, *c е константа
```

Указатели и масиви

В C++ има интересна и полезна връзка между указателите и масивите. Изразява се в това, че имената на масивите са указатели към техните "първи" елементи. Последното позволява указателите да се разглеждат като алтернативен начин за обхождане на елементите на даден масив.

Указатели и едномерни масиви

Нека a е масив, дефиниран по следния начин:
int a[100];
Тъй като a е указател към a[0], *a е стойността на a[0], т.е. *a и a[0] са два различни записа на стойността на първия елемент на масива. Тъй като елементите на масива са разположени последователно в паметта, a + 1 е адреса на a[1], a + 2 е адреса на a[2] и т.н. a + n-1 е адреса на a[n-1]. Тогава *(a+i) е друг запис на a[i] (i = 0, 1, ..., n-1).
Има обаче една особеност. Имената на масивите са константни указатели. Заради това, някои от аритметичните оператори, приложими над указатели, не могат да се приложат над масиви. Такива са ++, -- и присвояването на стойност. Използването на указатели е по-бърз начин за достъп до елементите на масива и заради това се предпочита. Индексираните променливи правят кода по-ясен и разбираем. В процеса на компилация всички конструкции от вида a[i] се преобразуват в *(a+i), т.е. операторът за индексиране [] се обработва от компилатора чрез адресна аритметика. Полезно е да отбележим, че операторът [] е ляво-асоциативен и с по-висок приоритет от унарните оператори (в частност от оператора за извличане на съдържание *).

Указатели и двумерни масиви

Името на двумерен масив е константен указател към първия елемент на едномерен масив от константни указатели. Ще изясним с пример казаното.
Нека a е двумерен масив, дефиниран по следния начин: int a[10][20];
Променливата a е константен указател към първия елемент на едномерния масив a[0], a[1], ..., a[9], като всяко a[i] е константен указател към a[i][0] (i = 0,1, ..., 9), т.е.
a

a[0] a[0][0] a[0][1] … a[0][19]

a[1] a[1][0] a[1][1] … a[1][19]

…

…

a[9] a[9][0] a[9][1] … a[9][19]

Тогава

**a == a[0][0]
a[0] == *a a[1] == *(a + 1) ... a[9] == *(a + 9),
т.е.
a[i] == *(a + i)
Като използваме, че операторът за индексиране е лявоасоциативен и с по-висок приоритет от оператора *, получаваме:
a[i][j] == (*(a+i))[j] == *(*(a+i)+j).
(вж. задачи за писмения в учебника)

Указатели и низове

Низовете са масиви от символи. Името на променлива от тип низ е константен указател, както и името на всеки друг масив. Така всичко, което казахме за връзката между масив и указател е в сила и за низ – указател.

Следващият пример илюстрира обхождане на низ чрез използване на указател към char. Обхождането продължава до достигане на знака за край на низ.

```
#include <iostream.h>
int main()
{char str[] = "C++Language"; // str е константен указател
 char* pstr = str;                  // pstr е указател към низа str
 while (*pstr)
 {cout << *pstr << '\n';
  pstr++;
 }                  // pstr вече не е свързан с низа “C++Language”.
 return 0;
}
```

Тъй като низът е зададен чрез масива от символи str, str е константен указател и не може да бъде променяна стойността му. Затова се налага използването на помощната променлива pstr. Ако низът е зададен чрез указател към char, както е в следващата програма, не се налага използването на такава.

```
#include <iostream.h>
int main()
{char* str = "C++Language";  // str е променлива
 while (*str)
 {cout << *str << '\n';
  str++;
 }
 return 0;
}
```

Примерите показват, че задаването на низ като указател към char има предимство пред задаването като масив от символи. Ще отбележим обаче, че дефиницията
char* str = "C++Language";
не може да бъде заменена с
char* str;
cin >> str;
следвани с въвеждане на низа “C++Language”(ще напомним, че не е възможно въвеждане на стойност на променлива от тип указател чрез оператора cin), докато дефиницията
char str[20];
позволява въвеждането му чрез cin, т.е.

cin >> str;
е допустимо.

Функции

Добавянето на нови оператори и функции в приложенията, реализирани на езика C++, се осъществява чрез функциите. Те са основни структурни единици, от които се изграждат програмите на езика. Всяка функция се състои от множество от оператори, оформени подходящо за да се използват като обобщено действие или операция. След като една функция бъде дефинирана, тя може да бъде изпълнявана многократно за различни входни данни.

Програмите на езика C++ се състоят от една или повече функции. Сред тях задължително трябва да има точно една с име main и наречена главна функция. Тя е първата функция, която се изпълнява при стартиране на програмата. Главната функция от своя страна може да се обръща към други функции. Нормалното изпълнение на програмата завършва с изпълнението на главната функция (Възможно е изпълнението да завърши принудително с изпълнението на функция, различна от главната).

Използването на функции има следните предимства:
- Програмите стават ясни и лесни за тестване и модифициране.
- Избягва се многократното повтаряне на едни и същи програмни фрагменти. Те се дефинират еднократно като функции, след което могат да бъдат изпълнявани произволен брой пъти.
- Постига се икономия на памет, тъй като кодът на функцията се съхранява само на едно място в паметта, независимо от броя на нейните изпълнения.

8.3 Дефиниране на функции

Синтаксис

Дефиницията на функция се състои от две части: заглавие (прототип) и тяло. Синтаксисът й е показан на Фиг. 8.4.

Дефиниране на функция

[<модификатор>]опц[<тип_на_функция>] опц <име_на_функция>
(<формални_параметри>)
{<тяло>
}
където
<модификатор>::= inline|static| ...
<тип_на_функцията> ::= <име_на_тип> | <дефиниция_на_тип>
<име_на_функция> ::= <идентификатор>
<формални_параметри> :: <празно> | void |
<параметър> {,
<параметър>}
<параметър> ::= <тип>[& |opc * [const]opc] opc <име_на_параметър>
<тип> ::= <име_на_тип>
<име_на_параметър> ::= <идентификатор>
<тяло> ::= <редица_от_оператори_и_дефиниции>

Дефиниция на функция

Модификаторите са спецификатори, които задават препоръка за компилатора (inline), класа памет (extern или static) и др. характеристики. Ще дадем примери в следващите разглеждания. Ако <модификатор> е пропуснат, подразбира се extern.
Типът на функцията е произволен тип без масив и функционален, но се допуска да е указател към такива обекти (в широкия смисъл на думата). Ако е пропуснат, подразбира се int.
Името на функцията е произволен идентификатор. Допуска се неоднозначност.
Списъкът от формални параметри (нарича се още сигнатура) може да е празен или void.
Например, следната функция извежда текст:

```
void printtext(void)
{cout << "C++ Programming Language \n"
        cout << "B. Stroustrup \n";
        return;
}
```

В случай, че списъкът е непразен, имената на параметрите трябва да са различни. Те заедно с името определят еднозначно функцията. Формалните параметри са: параметри – стойности, параметри – указатели и параметри – псевдоними. Името на параметъра се предшества от тип.

Примери:

```
int a, int const& b, double& x, int const * y, const int* a
```

Засега няма да използваме параметри, специфицирани със const.
Тялото на функцията е редица от дефиниции и оператори. Тя описва алгоритъма, реализиращ функцията. Може да съдържа един или повече оператора return.
Операторът return (Фиг. 8.5) връща резултата на функцията в мястото на извикването.

Оператор return

Синтаксис

return [<израз>]опц

където

- return е запазена дума;
- <израз> е произволен израз от тип <тип_на_функцията> или съвместим с него. Ако типът на функцията е void, <израз> се пропуска. В този случай е възможно и return да се пропусне.

Семантика

Пресмята се стойността на <израз>, конвертира се до типа на функцията (ако е възможно) и връщайки получената стойност в мястото на извикването на функцията, прекратява изпълнението ѝ.

Оператор return

Забележка: Ако функцията не е от тип void, тя задължително трябва да върне стойност. Това означава, че операторът return трябва да се намира във всички разклонения на тялото. В противен случай, повечето компилатори ще изведат съобщение или предупреждение за грешка. Възможно е обаче функцията да върне случайна стойност, което е лошо. По-добре е функцията да върне някаква безобидна стойност, отколкото случайна.

(„**Декларация на функция" - невключено в анотацията на въпроса**)

```
/*********************************************************/
```

Функциите могат да се дефинират в произволно място на програмата, но не и в други функции. Преди да се извика една функция, тя трябва да е "позната" на компилатора. Това става, като дефиницията на функцията се постави пред main или когато функцията се дефинира на произволно място в частта за дефиниране на функции, а преди дефинициите на функциите се постави само нейната декларация (Фиг. 8.6).

Декларация на функция

<декларация_на_функция> ::=
[<модификатор>][<тип_на_резултата>]<име_на_функция>
([<формални_параметри>]);

Декларация на функция

Възможно е имената на параметрите във <формални_параметри> да се пропуснат.

Семантика

Описанието на функция задава параметрите, които носят входа и изхода, типа на резултата, а също и алгоритъма, за реализиране на действията, което функцията дефинира. Параметрите-стойности най-често задават входа на функцията. Параметрите-указатели и псевдоними са входно-изходните параметри за нея. Алгоритъмът се описва в тялото на функцията. Изпълнението на функцията завършва при достигане на края на тялото или след изпълнение на оператор return [<израз>]опц;.
/**/

Обръщение към функция

Синтаксис
<обръщение_към_функция> ::=
<име_на_функция>()|
<име_на_функция>(void)|
<име_на_функция>(<фактически_параметри>)
където <фактически_параметри> са толкова на брой, колкото са формалните параметри. Освен по брой, формалните и фактическите параметри трябва да си съответстват по тип, по вид и по смисъл.

Съответствието по тип означава, че типът на i-тия фактически параметър трябва да съвпада (да е съвместим) с типа на i-тия формален параметър. Съответствието по вид се състои в следното: ако формалният параметър е параметър-указател, съответният му фактически параметър задължително е променлива или адрес на променлива, ако е параметър-псевдоним, съответният му фактически параметър задължително е променлива (за реализацията Visual C++, 6.0 от същия тип) и ако е параметър-стойност – съответният му фактически параметър е израз.

Семантика
Обръщението към функция е унарна операция с най-висок приоритет и с операнд - името на функцията. Последното пък е указател със стойност адреса на мястото в паметта където е записан програмният код на функцията. Ако функцията определя процедура, обръщението към нея се оформя като оператор (завършва с ;). Опитът за използването й като израз предизвиква грешка. Ако функцията връща резултат както

чрез return, така и чрез някой от формалните си параметри, обръщението към нея може да се разглежда и като оператор, и като израз. И ако функцията връща резултат единствено чрез оператора return, обръщението към нея има единствено смисъла на израз. Използването му като оператор не води до грешка, но не предизвиква видим резултат.
Обръщението към функция предизвиква генериране на нова стекова рамка и се осъществява на следните два етапа:

Свързване на формалните с фактическите параметри

За целта първият формален параметър се свързва с първия фактически, вторият формален параметър се свързва с втория фактически и т.н. последният формален параметър се свързва с последния фактически параметър. Свързването се реализира по различни начини в зависимост от вида на формалния параметър.

а) формален параметър – стойност

В този случай се намира стойността на съответния му фактически параметър. В стековата рамка на функцията за формалния параметър се отделя толкова памет, колкото типът му изисква и в нея се откопирва стойността на фактическия параметър.

б) формален параметър – указател

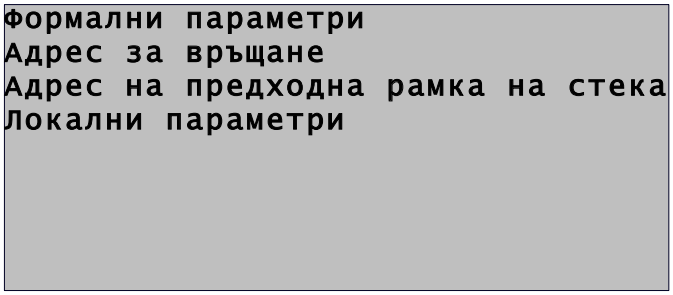
В този случай в стековата рамка на функцията за формалния параметър се отделят 4B, в които се записва стойността на фактическия параметър, която е адрес на променлива. Действията, описани в тялото се изпълняват със съдържанието на формалния параметър - указател. По такъв начин е възможна промяна на стойността на променливата, чийто адреа е предаден като фактически параметър.

в) формален параметър – псевдоним

Формалният параметър-псевдоним се свързва с адреса на фактическия. За него в стековата рамка на функцията памет не се отделя. Той просто “прелита” и се “закачва” за фактическия си параметър. Действията с него се извършват над фактическия параметър.

Изпълнение на тялото на функцията

Аналогично е на изпълнението на блок.
При всяко обръщение към функция в програмния стек се включва нов “блок” от данни. В него се съхраняват формалните параметри на функцията, нейните локални променливи, а също и някои “вътрешни” данни като return-адреса и др. Този блок се нарича стекова рамка на функцията.
В дъното на стека е стековата рамка на main. На върха на стека е стековата рамка на функцията, която се обработва в момента. Под нея е стековата рамка на функцията, извикала функцията, обработваща се в момента. Ако изпълнението на една функция завършва, нейната стекова рамка се отстранява от стека.
Видът на стековата рамка зависи от реализацията. С точност до наредба, тя има вида:



Стекова рамка

Масивите като формални параметри

Едномерни масиви
Съществуват различни начини за задаване на формални параметри от тип едномерен масив.
а) традиционен
Дефиницията
T a[]
където T е скаларен тип, задава параметър a от тип едномерен масив с базов тип T. Може да се укаже горна граница на масива, но компилаторът я пренебрегва.

Примери:
int a[] - a е параметър от тип масив от цели числа,
int a[10] - еквивалентна е на int a[],
double b[] - b е параметър от тип масив от реални числа,
char c[] - c е параметър от тип масив от символи.

б) чрез указател
Дефиницията
T* p
където T е скаларен тип, задава параметър p от тип указател към тип T. От връзката между масив и указател следва, че тази дефиниция може да се използва и за дефиниране на формален параметър от тип масив.

Примери: Следните дефиниции на формални параметри са еквивалентни на тези от примера по-горе:
int* a - a е параметър от тип указател към int
double* b - b е параметър от тип указател към double.
char* c - c е параметър от тип указател към char.

И в двата случая фактическият параметър се указва с името на едномерен масив от същия тип. Необходимо е също на функцията да се подаде като параметър и размерът на масива.
Забележки:

Функциите работят с направо с масива a, а не с негови копия. Промените на елементите на масива се запазват след излизане от функцията.
Размерът на масивът не може да се разбере от неговото описание. Затова се налага използването на допълнителния параметър m в списъка от аргументи на функциите. Последното не се отнася за масивите, представляващи символни низове, тъй като те завършват със знака за край на низ '\0'.

Многомерни масиви

Когато многомерен масив трябва да е формален параметър на функция, в описанието му трябва да присъстват като константи всички размери с изключение на първият. Например, декларацията

```
void readarr2(int n, int matr[][20]);
```

определя matr като двумерен масив (редица от двадесеторки от цели числа).
Описанието

```
int (*matr)[20]
```

е еквивалентно на
int matr[][20]
Скобите, ограждащи *matr, са задължителни. В противен случай, тъй като [] е с по-висок приоритет от *, int *matr[20] ще се интерпретира като "matr е масив с 20 елемента от тип *int".
(вж. допълнителни задачи за писмения в учебника)

10. Процедурно програмиране – основни информационни и алгоритмични структури (C++).

Основните принципи на структурното програмиране са принципа за **модулност** и принципа за **абстракция на данните**.Съгласно принципа за модулност, програмата се разделя наподходящи взаимосвързани части, всяка от които се реализирачрез определени средства. Целта е промените в представянето наданните да не променят голям брой от модулите на програмата. Съгласно принципа за абстракция на данните, методите за използване на данните се отделят от методите за тяхното конкретно представяне. Програмите се конструират така, че да работят с абстрактни данни – данни с неуточнено представяне. След това представянето се конкретизира с помощта на множество функции, наречени конструктори, мутатори и функции за достъп (селектори), които реализират абстрактните данни по конкретен начин. Така при решаването на даден проблем се оформят следните нива на абстракция:
1. Приложения в проблемната област.
2. Модули, реализиращи основните операции над данните.
3. Примитивни операции – конструктори, мутатори, селектори.
4. Избор на представянето на данните. Реализацията на подхода трябва да е такава, че всяко ниво използва единствено средствата на непосредствено следващото го ниво. По този начин, промените, които възникват на едно ниво ще се отразят само на предходното ниво. При изпълнение на компютърна програма се извършват определени действия над данните, дефинирани в програмата. Тези данни се съхраняват в **информационните структури**, допустими в съответния език за програмиране. Най-общо типовете информационни структури могат да бъдат разделени на два вида: вградени и абстрактни. Вградените типове са предварително дефинирани и се поддържат от самия език, а абстрактните типове се дефинират от програмиста. Друга класификация на типовете е следната: скаларни и съставни типове.

1.) Скаларните типове представят данни, които се състоят само от една компонента. При съставните типове данни, данните представляват редица от компоненти. Скаларните типове, поддържани в C++ са следните: булев, цял, реален, символен, изброен, указател, псевдоним. Съставните типове, поддържани в C++ са следните: масив, вектор, запис.

Логически тип

Нарича се още булев тип. Типът е стандартен, вграден в реализацията на C++. За означаването му се използва запазената дума bool (съкращение от boolean).

Множество от стойности

Дефинира се по обичайния начин.

Примери:

```
bool b1, b2;
bool b3 = false;
```

Дефиницията свързва булевите променливи с множеството от стойности на типа булев или с конкретна стойност от това множество като отделя по 1 байт оперативна памет за всяка от тях. Стойността на тази памет е неопределена или е булевата константа, свързана с дефинираната прменлива, в случай, че тя е инициализирана.

Оператори и вградени функции

Логически оператори

Оператор за логическо умножение (конюнкция)
Той е двуаргументен (бинарен) оператор. Означава се с and или && (за Visual C++)
Операторът се поставя между двата си аргумента. Такива оператори се наричат инфиксни.

Оператор за логическо събиране (дизюнкция)
Той също е бинарен, инфиксен оператор. Означава се с or или || (за Visual C++ 6.0)
Оператор за логическо отрицание

Той е едноаргументен (унарен) оператор. Означава се с not или ! (за Visual C++ 6.0)
Поставя се пред единствения си аргумент. Такива оператори се наричат префиксни.
Смисълът на операторите and, or и not е разширен чрез разширяване смисъла на булевите константи. Прието е, че true е всяка стойност, различна от 0 и че false е стойността 0.

Оператори за сравнение

Над булевите данни могат да се извършват следните инфиксни оператори за сравнение:
==, !=, >, >=, <, <=
Сравняват се кодовете.
Примери:
false < true e true
false > false e false
true >= false e true

Въвеждане

Не е възможно въвеждане на стойност на булева променлива чрез оператора >>, т.е. операторът
cin >> b1;
където b1 е булевата променлива, дефинирана по-горе, е недопустим.

Извеждане

Осъществява се чрез оператора
cout << <булева_константа>;
или по-общо
cout << <булев_израз>;
където синтактичната категория <булев_израз> е определена по-долу.
Извежда се кодът на булевата константа или кодът на булевата константа, която е стойност на <булев_израз>.
Булеви изрази

Булевите изрази са правила за получаване на булева стойност. Дефинират се рекурсивно по следния начин:
Булевите константи са булеви изрази.
Булевите променливи са булеви изрази.
Прилагането на булевите оператори not (!), and (&&), or (||) над булеви изрази е булев израз.

Прилагането на операторите за сравнение ==, !=, >, >=, <, <= към булеви изрази е булев израз.

Числени типове

Целочислени типове

Ще разгледаме целочисления тип int.

Типът е стандартен, вграден в реализацията на езика. За означаването му се използва запазената дума int (съкращение от integer).

Допълнение: Целите числа могат да са в десетична, осмична и шестнадесетична позиционна система. Осмичните числа започват с 0 (нула), а шестнадесетичните - с 0x (нула, x).

Елементите от множеството от стойности на типа int се наричат константи от тип int. Променлива величина, множеството от допустимите стойности, на която съвпада с множеството от стойности на типа int, се нарича цяла променлива или променлива от тип int. Дефинира се по обичайния начин. Дефиницията свързва променливата с множеството от стойности на типа int или с конкретна стойност от това множество като отделя по 4 байта (1 дума) оперативна памет за всяка от тях. Ако променливата не е била инициализирана, стойността на свързната с нея памет е неопределена, а в противен случай е указаната при инициализацията стойност.

Аритметични оператори

Унарни оператори

Записват се пред или след единствения си аргумент.

+, - са префиксни оператори. Потвърждават или променят знака на аргумента си.

Следните означения съдържат унарния оператор + или -:

-i +j -j +i -567

Бинарни оператори

Имат два аргумента. Следните аритметични оператори са инфиксни:

Оператор	Операция
+	събиране
-	изваждане
*	умножение
/	целочислено деление
%	остатък от целочислено деление

Логически оператори

Логическите оператори, реализиращи конюнкция, дизюнкция и отрицание, могат да се прилагат над целочислени константи. Дефинират се по същия начин, както при булевите константи, но целите числа, които са различни от 0 се интерпретират true, а 0 – като false.

Примери:

123 and 0 е false

0 or 15 e true
not 67 e false

Оператори за сравнение

Над цели константи могат да се прилагат следните инфиксни оператори за сравнение:

Оператор	Операция
==	сравнение за равно
!=	сравнение за различно
>	сравнение за по-голямо
>=	сравнение за по-голямо или равно
<	сравнение за по-малко
<=	сравнение за по-малко или равно

Наредбата на целите числа е като в математиката.

Вградени функции

В езика C++ има набор от вградени функции. Обръщението към такива функции има следния синтаксис:

<име_на_функция>(<израз>, <израз>, …, <израз>)

и връща стойност от типа на функцията.

Тук ще разгледаме само едноаргументната целочислена функция abs.

abs(x) – намира |x|, където x е цял израз

Примери:

abs(-1587) = 1587 abs(0) = 0 abs(23) = 23

Въвеждане

Реализира се по стандартния и разгледан вече начин.

Пример: Ако

int i, j;

операторът

cin >> i >> j;

въвежда стойности на целите променливи i и j.

Извеждане

Реализира се чрез оператора

cout << <цяла_константа>;

Други целочислени типове

Други цели типове се получават от int като се използват модификаторите short, long, signed и unsigned. Тези модификатори доопределят някои аспекти на типа int.

За реализацията Visual C++ 6.0 са в сила:

Тип	Диапазон	Необходима памет
short int	-32768 до 32767	2 байта
unsigned shortint	0 до 65535	2 байта
long int	-2147483648 до 2147483647	4 байта
unsigned long int	0 до 4294967295	4 байта
unsigned int	0 до 4294967295	4 байта

Запазената дума int при тези типове се подразбира и може да бъде пропусната. Типовете short int (или само short) и long int (или само long) са съкратен запис на signed short int и signed long int.

Реални типове

Ще разгледаме реалния тип double.

Типът е стандартен, вграден във всички реализации на езика.

Аритметични оператори

Унарни оператори

+, - Префиксни са. Потвърждават или променят знака на аргумента си.

Бинарни оператори

Имат два аргумента. Следните аритметичните оператори са инфиксни:

Оператор	Операция
+	събиране
-	изваждане
*	умножение
/	деление (поне единият аргумент е реален)

Логически оператори

Логическите оператори, реализиращи операциите конюнкция, дизюнкция и отрицание, могат да се прилагат над реални константи. Дефинират се по същия начин, както при булевите константи, като реалните числа, които са различни от 0.0 се интерпретират като true, а 0.0 – като false.

Оператори за сравнение

Над реални данни могат да се прилагат стандартните инфиксни оператори за сравнение (==, !=, >, >=, <, <=).

Допълнение: Сравнението за равно на две реални числа x и y се реализира обикновено чрез релацията: $|x - y| < \varepsilon$, където $\varepsilon = 10\text{-}14$ за тип double. По-добър начин е да се използва релацията:

Вградени функции

При цял или реален аргумент, следните функции връщат реален резултат от тип double:

Функция	Намира
sin(x)	Синус, sin x, x е в радиани
Cos(x)	косинус, cos x, x е в радиани
tan(x)	тангенс, tg x, x е в радиани
asin(x)	аркуссинус, arcsin $x \in [-\pi/2, \pi/2]$, $x \in [-1, 1]$

acos(x)	аркускосинус, arccos x ∈ [0, π], x ∈ [-1, 1]
atan(x)	аркустангенс, arctg x ∈ (-π/2, π/2)
exp(x)	експонента, ex
log(x)	натурален логаритъм, ln x, x > 0
log10(x)	десетичен логаритъм, lg x, x > 0
sinh(x)	хиперболичен синус, sh x
cosh(x)	хиперболичен косинус, ch x
tanh(x)	хиперболичен тангенс, th x
ceil(x)	най-малкото цяло ≥ x, преобразувано в тип double
Floor(x)	най-голямото цяло ≤ x, преобразувано в тип double
fabs(x)	абсолютна стойност на x, |x|
sqrt(x)	Корен квадратен от x, x ≥ 0
pow(x, n)	степенуване, xn (x и n са реални от тип double).

Въвеждане и Извеждане се Реализира по стандартния начин операторите cin и cout.
Други реални типове

В езика C++ има и друг реален тип, наречен float. Различава се от типа double по множеството от стойностите си и заеманата памет.
Множеството от стойности на типа float се състои от реалните числа от диапазона от –3.4*1038 до 3.4*1038. За записване на константите от този диапазон са необходими 4 байта ОП.

Структура от данни масив

Под структура от данни се разбира организирана информация, която може да бъде описана, създадена и обработена с помощта на програма.

За да се определи една структура от данни е необходимо да се направи:
- логическо описание на структурата, което я описва на базата на декомпозицията ѝ на по-прости структури, а също на декомпозиция на операциите над структурата на по-прости операции.
- физическо представяне на структурата, което дава методи за представяне на структурата в паметта на компютъра.

Елементите на структури, които се състоят от една компонента, се наричат прости, или скаларни. Структури от данни, компонентите на които са редици от елементи, се наричат съставни.
Структури от данни, за които операциите включване и изключване на елемент не са допустими, се наричат статични, в противен случай - динамични.
Логическо описание

Масивът е крайна редица от фиксиран брой елементи от един и същ тип. Към всеки елемент от редицата е възможен пряк достъп, който се осъществява чрез индекс. Операциите включване и изключване на елемент в/от масива са недопустими, т.е. масивът е статична структура от данни.

Физическо представяне

Елементите на масива се записват последователно в паметта на компютъра, като за всеки елемент на редицата се отделя определено количество памет.

В езика C++ структурата масив се реализира чрез типа масив.
Тип масив

В C++ структурата от данни масив е реализирана малко ограничено. Разглежда се като крайна редица от елементи от един и същ тип с пряк достъп до всеки елемент, осъществяващ се чрез индекс с цели стойности, започващи от 0 и нарастващи с 1 до указана горна граница. Дефинира се от програмиста.

Дефиниране на масив

Типът масив се определя чрез задаване на типа и броя на елементите на редицата, определяща масив. Нека T е име или дефиниция на произволен тип, различен от псевдоним, void и функционален. За типа T и константния израз от интегрален или изброен тип с положителна стойност size, T[size] е тип масив от size елемента от тип T. Елементите се индексират от 0 до size–1. T се нарича базов тип за типа масив, а size – горна граница.

Примери:

int[5] дефинира масив от 5 елемента от тип int, индексирани от 0 до 4;
double[10] дефинира масив от 10 елемента от тип double, индексирани от 0 до 9;
bool[4] дефинира масив от 4 елемента от тип bool, индексирани от 0 до 3.
Множество от стойности

Множеството от стойности на типа T[size] се състои от всички редици от по size елемента, които са произволни константи от тип T. Достъпът до елементите на редиците е пряк и се осъществява с помощта на индекс, като достъпът до първия елемент се осъществява с индекс със стойност 0, до последния – с индекс със стойност size-1, а до всеки от останалите елементи – с индекс със стойност с 1 по-голяма от тази на индекса на предишния елемент. Елементите от множеството от стойности на даден тип масив са константите на този тип масив.
Примери:
1. Следните редици {1,2,3,4,5}, {-3, 0, 1, 2, 0}, {12, -14, 8, 23, 1000} са константи от тип int[5].

2. Редиците {1.5, -2.3, 3.4, 4.9, 5.0, -11.6, -123.56, 13.7, -32.12, 0.98}, {-13, 0.5, 11.9, 21.98, 0.03, 1e2, -134.9, 0.09, 12.3, 15.6} са константи от тип double[10].
Примери:

```
int a[5];
double c[10];
bool b[3];
enum {FALSE, TRUE} x[20];
double p[4] = {1.25, 2.5, 9.25, 4.12};
```

Дефиницията

Примери:

Дефиницията
int q[5] = {1, 2, 3};
е еквивалентна на
int q[] = {1, 2, 3, 0, 0};
Дефиницията
double r[] = {0, 1, 2, 3};
е еквивалентна на

```
double r[4] = {0, 1, 2, 3};
```

Дефиницията с инициализация е един начин за свързване на променлива от тип масив с конкретна константа от множеството от стойности на този тип масив. Друг начин предоставят т.нар. индексирани променливи.
Всяка индексирана променлива е от базовия тип.

Синтаксис на индексираните променливи

Примери:
1. С променливата a, дефинирана по-горе, са свързани индексираните променливи a[0], a[1], a[2], a[3] и a[4], които са от тип int.
2. С променливата b са свързани индексираните променливи b[0], b[1],…, b[9], които са от тип double.
3. С променливата x са свързани индексираните променливи x[0], x[1],…, x[19], които са от тип enum {FALSE, TRUE}.

Дефиницията на променлива от тип масив не само свързва променливата с множеството от стойности на указания тип, но и отделя определено количество памет (обикновено 4B), в която записва адреса в паметта на първата индексирана променлива на масива. Останалите индексирани променливи се разполагат последователно след първата. За всяка индексирана променлива се отделя по толкова памет, колкото базовият тип изисква.
Операции и вградени функции

Не са възможни операции над масиви като цяло, но всички операции и вградени функции, които базовият тип допуска, са възможни за индексираните променливи, свързани с масива.

Пример: Нека

```
int a[5], b[5];
```

Недопустими са:

```
cin >> a >> b;
a = b;
```

а също a == b или a != b.

Операторът

```
cout << a;
```

е допустим и извежда адреса на a[0].
Някои приложения на структурата от данни масив са: Търсене на елемент в редица, Сортиране на редица, Сливане на редици (за повече инфо за писмения изпит - вж. учебника)
Друго приложение на типът масив е за представяне на символни низове в C++.
Разглежданите досега масиви се наричат едномерни. Те реализират крайни редици от елементи от скаларен тип. Възможно е обаче типът на елементите да е масив. В този случай се говори за многомерни масиви.

Многомерни масиви

Масив, базовият тип на който е едномерен масив, се нарича двумерен. Масив, базовият тип на който е двумерен масив, се нарича тримерен и т.н. На практика се използват масиви с размерност най-много 3.

Дефиниране на многомерни масиви

Нека T е име или дефиниция на произволен тип, различен от псевдоним, void и функционален, size1, size2, …, sizen (n>1 е дадено цяло число) са константни изрази от интегрален или изброен тип с положителни стойности. T[size1][size2] … [sizen] е тип n-мерен масив от тип T. T се нарича базов тип за типа масив.

Примери:

int [5][3] дефинира двумерен масив от тип int;

double [4][5][3] дефинира тримерен масив от тип double;

Синтаксис на многомерен масив

Примери:

```
int x[10][20];
double y[20][10][5];
int z[3][2] = {{1, 3},
{5, 7},
{2, 9}};
int t[2][3][2] = {{{1, 3}, {5, 7}, {6, 9}},
        {{7, 8}, {1, 8}, {-1, -4}}};
```

Дефиницията на променлива от тип многомерен масив не само свързва променливата с множеството от стойности на указания тип, но и отделя определено количество памет за разполагане на индексираните променливи, свързани с нея. Индексираните променливи се разполагат последователно по по-бързото нарастване на по-далечните си индекси. За всяка индексирана променлива се отделя толкова памет, колкото базовият тип изисква.

Забележка: Двумерните масиви разполагат в ОП индексираните си променливи по по-бързото нарастване на втория индекс. Това физическо представяне се нарича представяне по редове. Тези масиви могат да бъдат използвани за реализация и работа с матрици и др. правоъгълни таблици.

Важно допълнение: При работа с масиви трябва да се има предвид, че повечето реализации не проверяват дали стойностите на индексите са в рамките на границите, зададени при техните дефиниции. Тази особеност крие опасност от допускане на трудноотриваеми грешки.

Тип указател

Променлива, това е място за съхранение на данни, което може да съдържа различни стойности. Идентифицира се с дадено от потребителя име (идентификатор). Има си и тип. Дефинира се като се указват задължително типът и името й. Типът определя броя на байтовете, в които ще се съхранява променливата, а също и множеството от операциите, които могат да се изпълняват над нея. Освен това, с променливата е свързана и стойност – неопределена или константа от типа, от който е тя. Нарича се още rvalue. Мястото в паметта, в което е записана rvalue, се идентифицира с адрес, който се нарича адрес на променливата или lvalue. По-точно адресът е адреса на първия байт от множеството байтове, отделени за променливата.

Пример: Фрагментът

```
int i = 1024;
```

дефинира променлива с име i и тип int. Стойността й (rvalue) е 1024. i именува място от паметта (lvalue) с размери 4 байта, като lvalue е адреса на първия байт на това място.

Намирането на адреса на дефинирана променлива става чрез унарния префиксен дясно-асоциативен оператор & (амперсанд). Приоритетът му е същия като на унарните оператори +, -, !, ++, -- и др.

Оператор &

Синтаксис

&<променлива>

където <променлива> е вече дефинирана променлива.

Семантика

Намира адреса на <променлива>.

Пример: &i е адреса на променливата i и може да се изведе чрез оператора cout << &i;

Операторът & не може да се прилага върху константи и изрази, т.е. &100 и &(i+5) са недопустими обръщения. Не е възможно също прилагането му и върху променливи от тип масив, тъй като те имат и смисъла на константни указатели.

Адресите могат да се присвояват на специален тип променливи, наречени променливи от тип указател или само указатели.

Дефиниране

Нека T е име или дефиниция на тип. За типа T, T* е тип, наречен указател към T. T се нарича указван тип или тип на указателя.

Примери:

int* е тип указател към тип int;

enum {a, b, c}* е тип указател към тип enum {a, b, c}.

Множество от стойности

Състои се от адресите на данните от тип T, дефинирани в програмата, преди използването на T*. Те са константите на типа T*. Освен тях съществува специална константа с име NULL, наречена нулев указател. Тя може да бъде свързвана с всеки указател независимо от неговия тип. Тази константа се интерпретира като “сочи към никъде”, а в позиция на предикат е false.

Дефиниция на променлива от тип указател

Дефиницията

T* a, b;

е еквивалентна на

T* a;

T b;

т.е. само променливата a е от тип указател към тип T.

Примери: Дефиницията

int *pint1, *pint2;

задава два указателя към тип int, а

int *pint1, pint2;

- указател pint1 към int и променлива pint2 от тип int.

Дефиницията на променлива от тип указател предизвиква в ОП да се отделят 4B, в които се записва някакъв адрес от множеството от стойности на съответния тип, ако дефиницията е с инициализация и неопределено или NULL, ако дефиницията не е с инициализация. Този адрес е стойността на променливата от тип указател, а записаното на този адрес е съдържанието й.

Пример: В резултат от изпълнението на дефинициите

```
int i = 12;
int* p = &i;              // p е инициализирано с адреса на i
double *q = NULL; // q е инициализирано с нулевия указтел
double x = 1.56;
double *r = &x;         // r е инициализирано с адреса на x
```

Операции и вградени функции

Извличане на съдържанието на указател

Осъществява се чрез префиксния, дясноасоциативен унарен оператор *. * има приоритет на унарен оператор.

Оператор *
Синтаксис
*<променлива_от_тип_указател>
Семантика
Извлича стойността на адреса, записан в <променлива_от_тип_ указател>, т.е. съдържанието на <променлива_от_тип_ указател>.
Като използваме дефинициите от примера по-горе, имаме:
*p е 12// 12 е съдържанието на p
*r е 1.56 // 1.56 е съдържанието на r

Освен, че намира съдържанието на променлива от тип указател, обръщението *<променлива_от_тип_указател>
е променлива от тип T. (Всички операции, допустими за типа T, са допустими и за нея.

Като използваме дефинициите от примера по-горе, *p и *r са цяла и реална променливи, съответно. След изпълнение на операторите за присвояване

```
*p = 20;
*r = 2.18;
```

стойността на i се променя на 20, а тази на r – на 2.18.
Аритметични и логически операции

Променливите от тип указател могат да участват като операнди в следните аритметични и логически операции +, -, ++, --, ==, !=, >, >=, < и <=. Изпълнението на аритметични операции върху указатели е свързано с някои особености, заради което аритметиката с указатели се нарича още адресна аритметика. Особеността се изразява в т. нар. мащабиране. Ще го изясним чрез пример.

Да разгледаме фрагмента

```
int *p;
double *q;
...
p = p + 1;
q = q + 1;
```

Операторът p = p + 1; свързва p не със стойността на p, увеличена с 1, а с p + 1*4, където 4 е броя на байтовете, необходими за записване на данна от тип int (p е указател към int). Аналогично, q = q + 1; увеличава стойността на q не с 1, а с 8, тъй като q е указател към double (8 байта са необходими за записване на данна от този тип).
Общото правило е следното: Ако p е указател от тип T*, p+i е съкратен запис на p + i*sizeof(T), където sizeof(T) е функция, която намира броя на байтовете, необходими за записване на данна от тип T.
Въвеждане

Не е възможно въвеждане на данни от тип указател чрез оператора cin. Свързването на указател със стойност става чрез инициализация или оператора за присвояване.

Извеждане
Осъществява се по стандартния начин - чрез оператора cout.

Указатели и масиви

В C++ има интересна и полезна връзка между указателите и масивите. Изразява се в това, че имената на масивите са указатели към техните "първи" елементи. Последното позволява указателите да се разглеждат като алтернативен начин за обхождане на елементите на даден масив.

Указатели и едномерни масиви

Нека a е масив, дефиниран по следния начин:
int a[100];
Тъй като a е указател към a[0], *a е стойността на a[0], т.е. *a и a[0] са два различни записа на стойността на първия елемент на масива. Тъй като елементите на масива са разположени последователно в паметта, a + 1 е адреса на a[1], a + 2 е адреса на a[2] и т.н. a + n-1 е адреса на a[n-1]. Тогава *(a+i) е друг запис на a[i] (i = 0, 1, ..., n-1).
Има обаче една особеност. Имената на масивите са константни указатели. Заради това, някои от аритметичните оператори, приложими над указатели, не могат да се приложат над масиви. Такива са ++, -- и присвояването на стойност. Използването на указатели е по-бърз начин за достъп до елементите на масива и заради това се предпочита. Индексираните променливи правят кода по-ясен и разбираем. В процеса на компилация всички конструкции от вида a[i] се преобразуват в *(a+i), т.е. операторът за индексиране [] се обработва от компилатора чрез адресна аритметика. Полезно е да отбележим, че операторът [] е ляво-асоциативен и с по-висок приоритет от унарните оператори (в частност от оператора за извличане на съдържание *).

Указатели и двумерни масиви

Името на двумерен масив е константен указател към първия елемент на едномерен масив от константни указатели. Ще изясним с пример казаното.
Нека a е двумерен масив, дефиниран по следния начин: int a[10][20];
Променливата a е константен указател към първия елемент на едномерния масив a[0], a[1], ..., a[9], като всяко a[i] е константен указател към a[i][0] (i = 0,1, ..., 9), т.е.

a[i] == *(a + i)

Като използваме, че операторът за индексиране е лявоасоциативен и с по-висок приоритет от оператора *, получаваме:

a[i][j] == (*(a+i))[j] == *(*(a+i)+j).

(вж. задачи за писмения в учебника)

Указатели и низове

Низовете са масиви от символи. Името на променлива от тип низ е константен указател, както и името на всеки друг масив. Така всичко, което казахме за връзката между масив и указател е в сила и за низ – указател.

Следващият пример илюстрира обхождане на низ чрез използване на указател към char. Обхождането продължава до достигане на знака за край на низ.

```
#include <iostream.h>
int main()
{char str[] = "C++Language"; // str е константен указател
 char* pstr = str;                    // pstr е указател към низа str
 while (*pstr)
 {cout << *pstr << '\n';
  pstr++;
 }                    // pstr вече не е свързан с низа “C++Language”.
 return 0;
}
```

Тъй като низът е зададен чрез масива от символи str, str е константен указател и не може да бъде променяна стойността му. Затова се налага използването на помощната променлива pstr. Ако низът е зададен чрез указател към char, както е в следващата програма, не се налага използването на такава.

```
#include <iostream.h>
int main()
{char* str = "C++Language";  // str е променлива
 while (*str)
 {cout << *str << '\n';
  str++;
 }
 return 0;
}
```

Примерите показват, че задаването на низ като указател към char има предимство пред задаването като масив от символи. Ще отбележим обаче, че дефиницията

```
char* str = "C++Language";
```

не може да бъде заменена с

```
char* str;
cin >> str;
```

следвани с въвеждане на низа “C++Language”(ще напомним, че не е възможно въвеждане на стойност на променлива от тип указател чрез оператора cin), докато дефиницията

```
char str[20];
```

позволява въвеждането му чрез cin, т.е.

```
cin >> str;
```

е допустимо.

Функции

Добавянето на нови оператори и функции в приложенията, реализирани на езика C++, се осъществява чрез функциите. Те са основни структурни единици, от които се изграждат програмите на езика. Всяка функция се състои от множество от оператори, оформени подходящо за да се използват като обобщено действие или операция. След като една функция бъде дефинирана, тя може да бъде изпълнявана многократно за различни входни данни.

Програмите на езика C++ се състоят от една или повече функции. Сред тях задължително трябва да има точно една с име main и наречена главна функция. Тя е първата функция, която се изпълнява при стартиране на програмата. Главната функция от своя страна може да се обръща към други функции. Нормалното изпълнение на

програмата завършва с изпълнението на главната функция (Възможно е изпълнението да завърши принудително с изпълнението на функция, различна от главната).

Използването на функции има следните предимства:

- Програмите стават ясни и лесни за тестване и модифициране.
- Избягва се многократното повтаряне на едни и същи програмни фрагменти. Те се дефинират еднократно като функции, след което могат да бъдат изпълнявани произволен брой пъти.
- Постига се икономия на памет, тъй като кодът на функцията се съхранява само на едно място в паметта, независимо от броя на нейните изпълнения.

8.3 Дефиниране на функции

Синтаксис

Дефиницията на функция се състои от две части: заглавие (прототип) и тяло.

Дефиниция на функция

Модификаторите са спецификатори, които задават препоръка за компилатора (inline), класа памет (extern или static) и др. характеристики. Ще дадем примери в следващите разглеждания. Ако <модификатор> е пропуснат, подразбира се extern.
Типът на функцията е произволен тип без масив и функционален, но се допуска да е указател към такива обекти (в широкия смисъл на думата). Ако е пропуснат, подразбира се int.
Името на функцията е произволен идентификатор. Допуска се неоднозначност.
Списъкът от формални параметри (нарича се още сигнатура) може да е празен или void.
Например, следната функция извежда текст:

```
void printtext(void)
{cout << “C++ Programming Language \n”
        cout << “B. Stroustrup \n”;
        return;
}
```

В случай, че списъкът е непразен, имената на параметрите трябва да са различни. Те заедно с името определят еднозначно функцията. Формалните параметри са: параметри – стойности, параметри – указатели и параметри – псевдоними. Името на параметъра се предшества от тип.

Примери:

int a, int const& b, double& x, int const * y, const int* a

Засега няма да използваме параметри, специфицирани със const.
Тялото на функцията е редица от дефиниции и оператори. Тя описва алгоритъма, реализиращ функцията. Може да съдържа един или повече оператора return.
Операторът return (Фиг. 8.5) връща резултата на функцията в мястото на извикването.

Оператор return

Синтаксис

return [<израз>]опц

където

- return е запазена дума;
- <израз> е произволен израз от тип <тип_на_функцията> или съвместим с него. Ако типът на функцията е void, <израз> се пропуска. В този случай е възможно и return да се пропусне.

Семантика

Пресмята се стойността на <израз>, конвертира се до типа на функцията (ако е възможно) и връщайки получената стойност в мястото на извикването на функцията, прекратява изпълнението ѝ.

Оператор return

Забележка: Ако функцията не е от тип void, тя задължително трябва да върне стойност. Това означава, че операторът return трябва да се намира във всички разклонения на тялото. В противен случай, повечето компилатори ще изведат съобщение или предупреждение за грешка. Възможно е обаче функцията да върне случайна стойност, което е лошо. По-добре е функцията да върне някаква безобидна стойност, отколкото случайна.

Обръщение към функция

Синтаксис

<обръщение_към_функция> ::=
<име_на_функция>()|
<име_на_функция>(void)|
<име_на_функция>(<фактически_параметри>)

където <фактически_параметри> са толкова на брой, колкото са формалните параметри. Освен по брой, формалните и фактическите параметри трябва да си съответстват по тип, по вид и по смисъл.

Съответствието по тип означава, че типът на i-тия фактически параметър трябва да съвпада (да е съвместим) с типа на i-тия формален параметър. Съответствието по вид се състои в следното: ако формалният параметър е параметър-указател, съответният му фактически параметър задължително е променлива или адрес на променлива, ако е параметър-псевдоним, съответният му фактически параметър задължително е променлива (за реализацията Visual C++, 6.0 от същия тип) и ако е параметър-стойност – съответният му фактически параметър е израз.

Семантика

Обръщението към функция е унарна операция с най-висок приоритет и с операнд - името на функцията. Последното пък е указател със стойност адреса на мястото в паметта където е записан програмният код на функцията. Ако функцията определя процедура, обръщението към нея се оформя като оператор (завършва с ;). Опитът за използването ѝ като израз предизвиква грешка. Ако функцията връща резултат както чрез return, така и чрез някой от формалните си параметри, обръщението към нея може да се разглежда и като оператор, и като израз. И ако функцията връща резултат единствено чрез оператора return, обръщението към нея има единствено смисъла на израз. Използването му като оператор не води до грешка, но не предизвиква видим резултат.

Обръщението към функция предизвиква генериране на нова стекова рамка и се осъществява на следните два етапа:

Свързване на формалните с фактическите параметри

За целта първият формален параметър се свързва с първия фактически, вторият формален параметър се свързва с втория фактически и т.н. последният формален

параметър се свързва с последния фактически параметър. Свързването се реализира по различни начини в зависимост от вида на формалния параметър.

а) формален параметър – стойност

В този случай се намира стойността на съответния му фактически параметър. В стековата рамка на функцията за формалния параметър се отделя толкова памет, колкото типът му изисква и в нея се откопирва стойността на фактическия параметър.

б) формален параметър – указател

В този случай в стековата рамка на функцията за формалния параметър се отделят 4B, в които се записва стойността на фактическия параметър, която е адрес на променлива. Действията, описани в тялото се изпълняват със съдържанието на формалния параметър - указател. По такъв начин е възможна промяна на стойността на променливата, чийто адреа е предаден като фактически параметър.

в) формален параметър – псевдоним

Формалният параметър-псевдоним се свързва с адреса на фактическия. За него в стековата рамка на функцията памет не се отделя. Той просто “прелита” и се “закачва” за фактическия си параметър. Действията с него се извършват над фактическия параметър.

Изпълнение на тялото на функцията

Аналогично е на изпълнението на блок.

При всяко обръщение към функция в програмния стек се включва нов “блок” от данни. В него се съхраняват формалните параметри на функцията, нейните локални променливи, а също и някои “вътрешни” данни като return-адреса и др. Този блок се нарича стекова рамка на функцията.

В дъното на стека е стековата рамка на main. На върха на стека е стековата рамка на функцията, която се обработва в момента. Под нея е стековата рамка на функцията, извикала функцията, обработваща се в момента. Ако изпълнението на една функция завършва, нейната стекова рамка се отстранява от стека.

```
Формални параметри
Адрес за връщане
Адрес на предходна рамка на стека
Локални параметри
```

Видът на стековата рамка зависи от реализацията. С точност до наредба, тя има вида:

Стекова рамка

Масивите като формални параметри

Едномерни масиви

Съществуват различни начини за задаване на формални параметри от тип едномерен масив.

а) традиционен

Дефиницията

T a[]

където T е скаларен тип, задава параметър a от тип едномерен масив с базов тип T. Може да се укаже горна граница на масива, но компилаторът я пренебрегва.

Примери:

int a[] - a е параметър от тип масив от цели числа,

int a[10] - еквивалентна е на int a[],

double b[] - b е параметър от тип масив от реални числа,

char c[] - c е параметър от тип масив от символи.

б) чрез указател

Дефиницията

T* p

където T е скаларен тип, задава параметър p от тип указател към тип T. От връзката между масив и указател следва, че тази дефиниция може да се използва и за дефиниране на формален параметър от тип масив.

Примери: Следните дефиниции на формални параметри са еквивалентни на тези от примера по-горе:

int* a - a е параметър от тип указател към int

double* b - b е параметър от тип указател към double.

char* c - c е параметър от тип указател към char.

И в двата случая фактическият параметър се указва с името на едномерен масив от същия тип. Необходимо е също на функцията да се подаде като параметър и размерът на масива.

Забележки:

Функциите работят с направо с масива a, а не с негови копия. Промените на елементите на масива се запазват след излизане от функцията.

Размерът на масивът не може да се разбере от неговото описание. Затова се налага използването на допълнителния параметър m в списъка от аргументи на функциите. Последното не се отнася за масивите, представляващи символни низове, тъй като те завършват със знака за край на низ ‘\0’.

Многомерни масиви

Когато многомерен масив трябва да е формален параметър на функция, в описанието му трябва да присъстват като константи всички размери с изключение на първият. Например, декларацията

void readarr2(int n, int matr[][20]);

определя matr като двумерен масив (редица от двадесеторки от цели числа).

Описанието

int (*matr)[20]

е еквивалентно на

int matr[][20]

Скобите, ограждащи *matr, са задължителни. В противен случай, тъй като [] е с по-висок приоритет от *, int *matr[20] ще се интерпретира като “matr е масив с 20 елемента от тип *int”.

(вж. допълнителни задачи за писмения в учебника)